

Análisis léxico («lexing»)

El análisis léxico consiste en dividir una cadena de caracteres en objetos llamados «tokens». En el fichero ejercicio1 hay la implementación de un pequeño analizador léxico.

Ejecutando el código, el programa imprime por pantalla:

```
('ID', 'abc')
('SPACE', ' ')
('NUMBER', '123')
('SPACE', ' ')
('ID', 'cde')
('SPACE', ' ')
('NUMBER', '456')
```

1) *Modificar el fichero para que se ignoren los espacios.*

Si se cambia la variable «text» de la siguiente manera

```
text = 'abc 123 ! cde 456'
```

El programa imprime el siguiente error:

```
('ID', 'abc')
('NUMBER', '123')
Traceback (most recent call last):
  File "ex1.py", line 29, in <module>
    for tok in tokenize(text):
  File "ex1.py", line 25, in tokenize
    raise SyntaxError('Bad char %r' % text[index])
SyntaxError: Bad char '!'
```

2) *Modificar el fichero para que imprima un mensaje cada vez que encuentra un carácter desconocido.*

El problema de «tokenizar» se resuelve fácilmente utilizando herramientas como SLY. En el fichero analizador hay una base para la implementación de un analizador léxico de expresiones matemáticas. Por ejemplo para la entrada:

```
a = 3 + (4 * 5)
```

la salida debería ser:

```
Token(type='ID', value='a', lineno=1, index=0)
Token(type='ASSIGN', value=' = ', lineno=1, index=2)
Token(type='NUMBER', value='3', lineno=1, index=4)
Token(type='PLUS', value='+', lineno=1, index=6)
Token(type='LPAREN', value='(', lineno=1, index=8)
Token(type='NUMBER', value='4', lineno=1, index=9)
Token(type='TIMES', value='*', lineno=1, index=11)
Token(type='NUMBER', value='5', lineno=1, index=13)
Token(type='RPAREN', value=')', lineno=1, index=14)
```

y la salida de

```
a < b
```

```
a <= b
a > b
a >= b

a != b
```

es

```
Token(type='ID', value='a', lineno=2, index=12)
Token(type='LT', value='<', lineno=2, index=14)
Token(type='ID', value='b', lineno=2, index=15)
Token(type='ID', value='a', lineno=3, index=29)
Token(type='LE', value='<=', lineno=3, index=31)
Token(type='ID', value='b', lineno=3, index=34)
Token(type='ID', value='a', lineno=4, index=47)
Token(type='GT', value='>', lineno=4, index=49)
Token(type='ID', value='b', lineno=4, index=51)
Token(type='ID', value='a', lineno=5, index=64)
Token(type='GE', value='>=', lineno=5, index=66)
Token(type='ID', value='b', lineno=5, index=69)
Token(type='ID', value='a', lineno=6, index=82)
Token(type='ID', value='b', lineno=6, index=87)
Token(type='ID', value='a', lineno=7, index=100)
Token(type='NE', value='!=', lineno=7, index=102)
Token(type='ID', value='b', lineno=7, index=105)
```

3) Completar la implementación del fichero analizador.

El lenguaje SVG es un lenguaje de marcado para describir gráficos vectoriales. En el fichero svg hay una base para la implementación de un analizador léxico de SVG. Por ejemplo, para la entrada:

```
<rect x="10" y="20" width="100" height="50" fill="red" />
<rect x="30" y="20" width="100" height="50" fill="red">
</rect>
```

la salida debería ser tres tokens, uno para cada etiqueta de apertura y uno para la etiqueta de cierre, y cada token debería contener el nombre de la etiqueta y un diccionario con los atributos y sus valores. Por ejemplo, el token

```
Token(type='OPEN_TAG', value=('rect', \{'x': '10', 'y':
'20', 'width': '100', 'height': '50', 'fill': 'red'\}), lineno=1,
index=0)
```

representa la etiqueta de apertura de un rectángulo con sus atributos y valores.

4) Completar la implementación del fichero svg para que también reconozca etiquetas de apertura sin atributos, como los tres tipos de etiquetas. Buscar en la documentación de SLY para que sirva la función .begin()